

QuizMark — From Textbook PDF to Marked Test

A production AI assessment platform: generates questions from the book, runs timed quizzes on students' phones, and returns marks with written feedback — piloted across five classes, ~150 students.

Ankur Batta · Akkawat (Brian) Pipatupatum

Project report · July 2026 · Live in production on AWS

1. The problem

Every teacher knows the arithmetic of assessment. Setting a good test from a textbook takes hours; marking a hundred and fifty scripts takes evenings; and by the time feedback reaches a student, the class has moved two chapters on. The feedback that matters most arrives too late to matter. QuizMark began with a simple question: if the textbook is already sitting there as a PDF, why can't it draft the paper, deliver the test, and pre-mark the routine answers under the teacher's eye — honestly, cheaply, and fast enough that a student sees feedback while the topic is still fresh?

~150 students across 5 classes in the live pilot	438 automated tests guarding grading correctness	3 AI providers — failover keeps marking alive	90.3% of the book captured — maths, tables, figures included	0 LLM calls to mark objective & high-confidence answers
---	---	--	--	--

2. Teaching the machine the textbook

The answer begins the moment an instructor uploads a book. A pipeline of background workers takes the PDF apart: text is chunked and embedded, mathematics is converted to LaTeX, and vision models read the tables and figures that plain text extraction would lose; scanned pages fall back to OCR. One index would blur mathematics into prose, so there are five — text plus specialist indexes for maths, tables, figures, and the book's own end-of-chapter exercises — vectors held in MongoDB itself: **no separate vector database to run, back up, or pay for**. This is the project's economic foundation: **the book is paid for once and consulted forever**. Ingestion is checkpointed (a crash or redeploy resumes without re-reading a page), and caches for page descriptions, LaTeX conversions, and validation verdicts ensure nothing is paid for twice. From then on, every AI call retrieves only the few passages it actually needs — a few hundred grounded tokens instead of a whole chapter. That is what retrieval-augmented generation (RAG) really buys: **roughly an order-of-magnitude cut in token spend**, repeated across thousands of calls, and prompts grounded so well that inexpensive models (gpt-4o-mini, Haiku-class) do work that would otherwise demand frontier-model prices. Grounding also keeps generated questions on-syllabus, so far fewer are rejected and regenerated — and when the validator does reject one, a critic-repair pass armed with retrieval and, when configured, web search fixes it rather than discarding it. The validator's rejection rules are injected verbatim into the generation prompt as static steering, so the model self-censors against the exact checklist the gate will enforce — before anything is spent.

3. Test day

An instructor picks a chapter and generates questions — multiple choice, true/false, or subjective, with difficulty control, some built directly on the book's own tables and figures so the test feels like the course rather than a chatbot. After review, the questions become a timed quiz, and **students scan a QR code and sit the test on their own phones**. This is where classroom reality bites: an entire class behind a single college NAT IP looks, to a naive server, like one abusive client. Rate limiting had to be rebuilt NAT-aware — the client's true IP read from a trusted position in the forwarding chain, defeating spoofed headers, with per-account brute-force lockout beneath a high per-IP ceiling so a shared campus address never locks the room out. A unique **(student, question, quiz)** database index makes a second submission fail atomically even under a concurrent double-tap, and deadlines are enforced server-side. While a classful of phones submits concurrently, the instructor watches results stream in live.

4. Marking as triage — most answers never touch an LLM

The marking pipeline is designed like hospital triage: **human judgment is reserved for the answers that genuinely need it**. Objective answers are checked against the answer key — deterministic, instant, free. Every subjective answer then meets a local pre-scorer: rubric keyword coverage blended with embedding similarity against the model answer. Only answers clearing a deliberately strict dual gate (**semantic similarity ≥ 0.92 and keyword coverage ≥ 0.60**) take full credit without further review — a conservative one-way door that prefers spending money over risking a mismatch, so every partial or ambiguous case goes onward to a RAG-grounded LLM that marks against the rubric with the book open and writes feedback. When confidence is lowest, the system retrieves wider context and raises a flag for a human. Flags focus the instructor's time on that uncertain minority — the rest is already marked — and audit-trailed one-click overrides keep the final word human. Analytics close the loop honestly: answer-key marks and AI marks are reported separately, over full populations with real thresholds, so the instructor is never shown a flattering but misleading number. **Authority over marks stays with the teacher throughout**: AI scores are labelled as such, every override is recorded, and the least certain judgments are routed to a person rather than a model by design.

5. Running it in production

None of this matters if the platform falls over mid-exam. The stack — **14 Docker services, 8 dedicated Celery workers over a Redis broker** — runs on a single AWS EC2 instance behind Caddy HTTPS, with every application and database port bound to 127.0.0.1 so only the reverse proxy is internet-reachable. Shipping an update is one command: the tree is rsync'd over SSH, API keys ride across in a mode-600 file that is sourced and then shredded — never on a command line — and containers start in health order (the API waits for MongoDB and Redis; the frontend waits for the API). Nothing is trusted to look fine: a **16-check verification suite runs automatically after every deploy**, testing the anti-double-submission indexes, the rate-limit proxy

configuration, the provider failover chain, and the freshness and gzip-integrity of last night's backup — which runs nightly, is integrity-checked, copied off-box to S3, and drilled with a real restore. A three-provider failover chain (OpenAI → Anthropic → Gemini) rotates per request — and it has fired for real, carrying marking through exhausted credits mid-pilot.

6. Proof: 150 students, 438 tests

This system has been used for real tests, not demos. Engineering rigour came first: **438 automated backend tests** cover grading correctness, concurrency races, security scoping, and analytics honesty; retrieval quality is measured against a hand-built golden query set (**mean reciprocal rank 0.84**); ingestion coverage is measured, not assumed, at **90.3% of a full textbook**. Then came the classroom: an instructor piloted QuizMark end-to-end with five classes of roughly thirty students each — about 150 in total — who took their tests through it, joining timed quizzes from their own phones, submitting under enforced deadlines, and receiving marks and written feedback within moments. Class after class, the instructor worked the review queue, overrides, live results, and PDF marks summaries on genuine student work. Instructor and students both responded positively: students to instant results and effortless phone joining; the instructor to the fairness of the flag-and-review workflow. The pilot shaped the product directly — the NAT and concurrency hardening exist because real students found the limits.

7. What it took

The hardest problems were about trust, not features. Early on, fluent-but-wrong answers over-scored on similarity alone — the cure was making full credit a strict dual gate and routing every doubtful case to the careful, rubric-grounded path. Malformed JSON and LaTeX from language models could silently zero a student's mark; hardened three-stage parsing and deterministic numeric checks (the model only *translates* a calculation to Python — a sandboxed evaluator computes it, so the number never originates from the model) took one live verification run from 7/9 correct to 9/9. Iterative audits hunted down misleading statistics — sampling bias, inflated averages, wrong denominators — and enforced per-instructor data isolation, sanitised LLM-generated HTML against stored XSS, and role-gated administration, because a marking system is only as good as the honesty of what it reports.

8. Technology at a glance

Frontend	Next.js (React, TypeScript), Tailwind CSS, KaTeX mathematics; dedicated mobile quiz player with QR join.
Backend	FastAPI (async Python); JWT auth with admin / instructor / student roles; NAT-aware rate limiting; bcrypt run off the event loop.
Data & AI	MongoDB as primary + vector store; RAG with four specialist indexes and reciprocal-rank fusion; per-capability failover (marking OpenAI → Anthropic → Gemini; embeddings Gemini → OpenAI); vision models and Tesseract OCR.
Infrastructure	Docker Compose (14 services, 8 Celery workers, Redis) on AWS EC2; Caddy HTTPS; one-command rsync deploys; 16-check post-deploy verification; nightly verified backups to S3.

9. Where it goes next

The response data now accumulating enables the next chapter: item analysis to calibrate question difficulty and adaptive quizzes matched to each student; a locally hosted small language model fine-tuned with LoRA adapters on the pilot's accumulated marking decisions (instructor overrides serve as ready-made training labels), taking over the mid-confidence marking tier and pushing marginal cost per answer toward zero while keeping student data on-premises; LMS integration (Moodle, Google Classroom) and institutional dashboards; handwritten answer-sheet OCR to bring offline exams into the same pipeline; multilingual textbooks; and cohort-wide answer-similarity detection for academic integrity.

10. Conclusion

The question that opened this report has an answer. A textbook PDF can draft the paper, deliver the test, and pre-mark the answers — with the teacher keeping the final word — and roughly 150 students have sat real tests that prove it. Every layer was engineered to a budget: retrieval instead of long contexts, cheap models where grounding permits, zero-cost deterministic paths wherever possible. Building it demanded the complete arc of a production AI system — ingestion pipelines, retrieval, cost-aware model routing, distributed workers, security auditing, cloud deployment — validated where it counts: in a classroom, giving educators their evenings back and students feedback while the topic is still fresh, at a running cost small enough for a single classroom budget.